

IC4302 - Proyecto 02: Products Search V2

Curso: Bases de Datos II (IC4302)

Semestre: Segundo Semestre 2025

Institución: Tecnológico de Costa Rica – Escuela de Ingeniería en Computación

VIDEO INFORMATIVO

[Products Search V2](#)

Instrucciones de Ejecución

► Desplegar información

1.1 Requisitos Previos

- Cuenta en Docker Hub: Es un sitio web donde puedes guardar y compartir imágenes de programas listos para usar. Es como una "nube" para aplicaciones.
- Docker y Docker Compose: Docker es una herramienta que permite ejecutar programas en "contenedores", que son como cajas que traen todo lo necesario para que el programa funcione igual en cualquier computadora. Docker Compose ayuda a iniciar varios de estos programas juntos fácilmente.
- Kubernetes (Minikube o Docker Desktop): Kubernetes es una plataforma que ayuda a administrar y ejecutar muchos contenedores a la vez, ideal para proyectos grandes. Minikube y Docker Desktop son formas sencillas de usar Kubernetes en tu propia computadora.
- Helm Charts instalados: Helm es una herramienta que facilita la instalación y actualización de aplicaciones en Kubernetes, usando "charts" que son como recetas pre-hechas.
- Git: Es una herramienta para guardar y controlar los cambios en el código de un proyecto, permitiendo trabajar en equipo y mantener un historial de versiones.
- Lens: Es un programa con interfaz gráfica que permite ver y administrar fácilmente los recursos y servicios que se están ejecutando en Kubernetes.

1.2 Instalación de Componentes

1. Descargue el repositorio del proyecto en su computadora

```
git clone <URL_REPO>
```

Después ingrese a la carpeta del repositorio por medio de la terminal bash:

```
cd 2025-02-IC4302
```

2. Construya la imagenes de docker

Para poder realizar la construcción de las imágenes Docker. debe ingresar a la carpeta **docker** desde una terminal Bash y ejecutar el siguiente comando:

```
./build.sh usuario
```

NOTA: Sustituya la palabra usuario con su usario de Docker Hub

3. Configure el registro de las imágenes para el chart

En su proyecto, ingrese a la carpeta de charts --> app --> templates --> values.yaml y registre el nombre de usuario en docker hub que desea utilizar en el campo **docker_registry**

```
config:
  docker_registry: SU_USUARIO # docker registry replace with your own username
  producer:
    enabled: true
```

4. Instale el Helm Chart del proyecto

En su proyecto, ingrese a la carpeta de **charts** desde una terminal Bash y ejecute el siguiente comando:

```
./install.sh
```

5. Desinstalación del Helm Chart del proyecto

En caso de que usted necesite hacer la desinstalación del helm chart, ingrese a la carpeta de **charts** desde una terminal Bash y ejecute el siguiente comando:

```
./uninstall.sh
```

NOTA: Si no necesita la instalación, ignore este paso

6. Como ingresar a la página WEB

► Desplegar información

Manual de acceso a la página web

Nuestro sitio web trata sobre cursos donde el usuario puede consultar todos los cursos disponibles, ver información como el precio, idioma, categoría, entidades, etc. Además, puede hacer búsquedas por filtros o por texto, donde obtendrá un highlight de los cursos con las palabras utilizadas.

Para acceder a **Products Search V2**, simplemente ingrese a <https://fullstack-app-ruby.vercel.app/>

A continuación, se explicará como utilizar la web. Esta página está hecha por Next.js, utiliza Firebase como método de autenticación y seguridad para verificación de tokens. Se puede dividir en cuatro módulos:

► 1. Autenticación:

Registrarse: Si es la primera vez utilizando la aplicación web, se debe crear una cuenta. Unicamente debe ingresar un correo electrónico valido y la contraseña, se debe confirmar para verificar que sea la misma.



Iniciar Sesión: Si ya tiene una cuenta registrada, unicamente debe colocar el correo y contraseña utilizados al crear la cuenta.



► 2. Cursos Generales:

Página Home: Esta es la página principal, donde el usuario es redirigido una vez inicia sesión. En esta pantalla puede ver todos los cursos disponibles, con su información general, como el título, descripción, imagen, rating, entre otros. Hay una paginación de 20 cursos, para una mejor experiencia de usuario. Además, el usuario puede buscar textualmente lo que busca y aparecerá la palabra en highlight, también puede usar la búsqueda avanzada para ver cursos con esas opciones.   

► 3. Detalles sobre Curso Específico:

Información de un curso: Esta es la página que se obtiene al darle click a un curso. En esta pantalla puede ver toda la información de este, para que tome la decisión si desea empezar el curso. Si se busco una palabra en especial, acá se va a mantener el highlighting. Además, también aparecen los cursos relacionados, para que el usuario siga navegando sobre los cursos que se ofrecen.  

► 4. Cerrar sesión:

Navbar: Si ya desea cerrar sesión por seguridad, se debe tocar el ícono del perfil en el header. Este ícono contiene la primera inicial de su correo electrónico. Al presionarlo obtendrá la opción para salir. Esto lo redigira a la pantalla de iniciar sesión. 

Pruebas Unitarias

► Desplegar información

Controller

► Desplegar información

Estas pruebas verifican el comportamiento principal del componente que controla la ingesta de archivos desde S3, el cálculo de MD5, y la publicación a RabbitMQ.

- **build_doc:** Se prueba que el documento generado incluya todos los campos correctos usando el key, el tamaño, el MD5 y la fecha simulada.
- **build_message:** Revisa que el mensaje solo contenga el id basado en el _id del documento.

- **list_s3**: Simula paginación de S3 y verifica que la función devuelva todos los objetos encontrados en varias páginas.
- **calculate_md5**: Se simula el Body que devuelve S3 y se verifica que el MD5 calculado sea correcto usando chunks de bytes.
- **test_publish_new**: Cuando un archivo no existe en Mongo, hace upsert y guarda el estado "new". Publica el mensaje en RabbitMQ.
- **test_publish_modified**: Cuando el archivo sí existe pero el MD5 cambió, actualiza la entrada y guarda el estado "modified". Publica el mensaje en RabbitMQ.
- **test_publish_unchanged**: Cuando el archivo existe y el MD5 es igual, no actualiza nada, no publica nada y devuelve unchanged.

```
test.py::test_build_doc PASSED
[ 14%]
test.py::test_build_message PASSED
[ 28%]
test.py::test_list_s3 PASSED
[ 42%]
test.py::test_calculate_md5 PASSED
[ 57%]
test.py::test_publish_new PASSED
[ 71%]
test.py::test_publish_modified PASSED
[ 85%]
test.py::test_publish_unchanged PASSED
[100%]

===== 7 passed in 0.43s =====
```

Web Scraper

► Desplegar información

- **test_obtenerProductos**: Verifica que la función retorne exactamente el contenido del HTML cuando la petición es exitosa.
- **test_obtenerLinks**: Esta prueba garantiza que el filtro de URLs es correcto.
- **test_obtenerLinks_vacio**: Confirma un buen manejo de entradas inválidas.
- **test_descargarHtml**: Esta prueba asegura la correcta creación y escritura de los archivos descargados.

```
WebScraper/test.py::test_obtenerProductos PASSED [ 25%]
WebScraper/test.py::test_obtenerLinks PASSED [ 50%]
WebScraper/test.py::test_obtenerLinks_vacio PASSED [ 75%]
WebScraper/test.py::test_descargarHtml PASSED [100%]

===== 4 passed in 0.25s =====
```

Beautiful Soup

► Desplegar información

- **test_parse_author**: Verifica que la función identifique correctamente el nombre del autor del curso y separe el comentario en distintos casos, incluso cuando el bloque contiene encabezados o palabras clave que deben ser limpiadas con soup.
- **test_extract_course_data_json_ld**: Comprueba que el componente pueda leer y procesar correctamente los datos estructurados en formato JSON-LD desde el html, extrayendo y parseando los campos solicitados.
- **test_extract_course_data_fallbacks**: Asegura que el sistema utilice correctamente los fallbacks al obtener y parsear los datos si no se obtienen en primera instancia.
- **test_download_html_from_s3**: Valida que la función descargue correctamente el HTML desde S3, confirmando recibir la respuesta correcta.
- **test_process_message**: Prueba el funcionamiento del flujo al recibir un mensaje, para actualizar mongo, generar y subir el json al volumen compartido y publicar la información en RabbitMQ para el siguiente componente.

```
test.py::test_parse_author PASSED [ 20%]
test.py::test_extract_course_data_json_ld PASSED [ 40%]
test.py::test_extract_course_data_fallbacks PASSED [ 60%]
test.py::test_download_html_from_s3 PASSED [ 80%]
test.py::test_process_message PASSED [100%]

===== 5 passed in 1.33s =====
```

Spacy Entity Extractor

► Desplegar información

Estas pruebas verifican el comportamiento principal del componente que usa spaCy para extraer entidades, actualizar el estado en MongoDB y escribir los archivos nuevos en el volumen compartido.

- **test_path_from_key**: Verifica que a partir del s3_key se genere correctamente la ruta del archivo JSON en la carpeta augmented.
- **test_entities_started**: Comprueba que entities_status actualice el documento en MongoDB con el estado "started" usando el _id correcto.
- **test_entities_error**: Valida que cuando el estado es "error", se guarde tanto entitiesExtraction como entitiesExtractionError en la colección.
- **test_normalize_text**: Revisa que normalize_text limpie saltos de línea y espacios múltiples, dejando el texto en una sola línea con espacios simples. También se prueba el comportamiento con texto vacío o None.
- **test_extract_entities**: Simula el modelo de spaCy para comprobar que extract_entities usa NLP internamente, devuelva una lista de diccionarios con type y value y elimine entidades duplicadas.

- `test_ensure_volume`: Valida que `ensure_volume` cree las carpetas `raw` y `augmented` dentro del `BASE_PATH` que se le pasa.
- `test_ensure_volume_empty`: Comprueba que, si el `base_path` es vacío, la función lance un `ValueError`.
- `test_message_flujo`: Prueba el flujo completo de `message`: Usa un JSON de entrada en la carpeta `raw`, construye el texto a partir de los campos del curso y las reseñas, llama a `extract_entities`, escribe el archivo nuevo en `augmented` con el campo `entities` y por ultimo, actualiza MongoDB dos veces: primero con `"started"` y al final con `"completed"`.

```
test.py::test_path_from_key PASSED
[ 12%]
test.py::test_entities_started PASSED
[ 25%]
test.py::test_entities_error PASSED
[ 37%]
test.py::test_normalize_text PASSED
[ 50%]
test.py::test_extract_entities PASSED
[ 62%]
test.py::test_ensure_volume PASSED
[ 75%]
test.py::test_ensure_volume_empty PASSED
[ 87%]
test.py::test_message_flujo PASSED
[100%]

===== 8 passed in 6.08s
=====
```

Spark Processor Job

► Desplegar información

Para validar las transformaciones de Spark sin tocar la escritura en Mongo Atlas se añadió un conjunto de pruebas unitarias en `Services/docker/SparkProcessorJob/app/test.py`. Ejecutan un `SparkSession` local y cubren:

- `test_uppercase_first_letter_capitalizes_string_columns`: verifica que las columnas texto de primer nivel se conviertan a *Title Case* sin alterar tipos numéricos ni `None`.
- `test_normalize_entities_handles_nested_structures`: comprueba la capitalización dentro de arreglos y estructuras anidadas (`entities`, `comentarios`, `tags`).
- `test_format_dates_ddmmyyyy_sql_formats_dates`: asegura que `date_extracted` acepte variantes (`YYYY/MM/DD`, `YYYY-MM-DD`) y se normalice a `DD/MM/YYYY` preservando nulos.
- `test_summary_generates_short_description`: valida la creación de `short-description`, truncando textos largos con sufijo `...` y manteniendo cadenas vacías para valores nulos.
- `test_add_related_products_builds_related_list`: garantiza que las entidades comunes generen recomendaciones sin duplicados ni auto-referencias.

Resultado más reciente

```
=> [8/8] RUN pytest -s -vv test.py && sleep 20
21.2s
=> => # PASSED
=> => # test.py::test_normalize_entities_handles_nested_structures PASSED
=> => # test.py::test_format_dates_ddmmyyyy_sql_formats_dates PASSED
=> => # test.py::test_summary_generates_short_description PASSED
=> => # test.py::test_add_related_products_builds_related_list PASSED
=> => # ===== 5 passed in 18.76s
=====
```

Configuración de componentes

► Details

Desplegar información

Controller

► Desplegar información

El Controller es el primer componente del pipeline y actúa como coordinador general del sistema. Su función principal es revisar periódicamente los archivos HTML almacenados en AWS S3 y decidir cuáles deben ser procesados.

Sus funciones principales son:

- Busca archivos HTML en carpetas específicas del bucket de AWS S3
- Compara los archivos usando hash MD5 para saber si son nuevos o han sido modificados
- Guarda información de cada archivo como el nombre, tamaño, estado en la base de datos, específicamente en la colección ingestion dentro de Mongo
- Envía mensajes a RabbitMQ solo para archivos nuevos o modificados, iniciando el procesamiento para que el siguiente componente pueda utilizarlo.

Este componente se ejecuta como un CronJob de Kubernetes, esto quiere decir que tiene ejecución automática dependiendo de la configuración establecida, en este caso se ejecuta cada hora, pero puede ser adaptado según la necesidad.

```
controller:
  enabled: true
  replicas: 1
  name: controller
  image: controller
  schedule: "0 * * * *" #every hour
```

Web Scraper

► Desplegar información

Para poder utilizar este componente, descargar localmente las librerías requests, selenium y dotenv. Además, configurar el .env con las credenciales de aws de ser necesario. Solo hay que correr el main una vez.

El web scraper es un pequeño código en python que se corre localmente fuera de kubernetes. Se encarga de recorrer la página de cursos online "edutin". Recorre las categorías "programacion", "cocina", "creativo", "salud", "negocio", "deporte", "psicologia", "ciencia", "cloud computing", "mantenimiento", "moda", "arte", "idiomas" y "marketing". Para esto, se usa selenium para hacer scroll en la página principal de edutin, y una vez que se hayan cargado suficientes cursos, se recupera el html de la página. Luego, se recorre el html en busca de la dirección que lleva a la información específica de cada curso. Cuando se encuentra, se extrae el html de esa dirección. Los html se descargan localmente en la carpeta "productos", y van numerados del 001 al 539. Una vez se han descargado los 505 cursos, estos se suben a la carpeta del bucket de aws, ic-tec-dataset/CARPETA_HCDCP/. Desde ahí se podrán recuperar los datos posteriormente.

Beautiful Soup

► Desplegar información

El BeautifulSoup Parser es el segundo componente del pipeline y es quien parsea la información extraída en los html para generar un json limpio con la información. Este componente actúa mediante RabbitMQ, escuchando los documentos publicados desde el controller para descargarlos, y avisando al Spaci Entity cuando genera un json. Sus funciones principales son:

- Escuchar RabbitMQ para leer los mensajes publicados por el Controller.
- Descargar el archivo html del bucket de S3 con la ruta dada por el Controller.
- Parsea el archivo, obteniendo toda la información revelante con la biblioteca BeautifulSoup
- Guarda la información en un archivo json dentro de la carpeta raw en el almacenamiento compartido entre pods
- Actualiza la colección "Ingestion" en mongo, poniendo en el campo "processing" el estado "started" cuando se recibe el mensaje y se empieza a parsear, y el estado "completed" cuando se almacena el json.
- Publica un mensaje en la cola de RabbitMQ compartida con el Spacy Entity Extractor, con el id del archivo y la ruta del json parseado.

Este componente se ejecuta como un Deployment de Kubernetes, pasa escuchando y ejecutándose según los mensajes que recibe del controller. Se recomienda mínimo utilizar 2 réplicas(beautifulSoup.yaml).

Como información útil, si se desea ver los json subidos al volumen compartido por este componente, se puede usar el siguiente comando:

```
kubectl exec -it <nombre del pod> -- bash
```

reemplazando con el nombre del pod del componente en su equipo. Dentro del pod, si desea ver un archivo por ejemplo el 251, puede usar este comando:

```
cat /app/data/raw/curso_251.json
```


Spacy Entity Extractor

► Desplegar información

El Spacy Entity Extractor es el tercer componente del pipeline y se encarga de analizar los textos procesados por BeautifulSoup para identificar y extraer entidades importantes como nombres de productos, organizaciones, fechas, y otros datos relevantes.

Este componente trabaja de forma continúa escuchando mensajes enviados por RabbitMQ y realiza las siguientes tareas:

- Escucha mensaje sde RabbitMQ que indican que el archivo está listo para procesar.
- Obtiene los archivos JSON de la carpeta raw que se encuentra en el disco compartido del proyecto y que fue creada por el componente de BeautifulSoup.
- Usa el modelo de Spacy y su función NER (Named Entity Recognition) para identificar entidades en diferentes campos establecidos en los documentos obtenidos como.
- Después de crear las entidades, crea nuevos archivos JSON y los almacena en la carpeta augmented para que sean procesados posteriormente por el componente de Spark
- Por último, actualiza la base de datos estableciendo que el procesamiento de entidades ha finalizado y llevar un control de trazabilidad sobre los documentos.

Este componente se ejecuta como un deployment en kubernetes con un mínimo de dos replicas para aumentar la velocidad de procesamiento, a diferencia del controller, este componente está siempre activo y esperando por nuevos mensajes.

```
controller:
  enabled: true
  replicas: 1
  name: controller
  image: controller
  schedule: "0 * * * *" #every hour
```

Para la implementación del modelo de Spacy se elige el modelo *es_core_news_lg* el cual está diseñado para manejar el idioma español y se utiliza su versión large, la cual tiene mucho mayor alcance a la hora de hacer la extracción de entidades. Algunas de las entidades que reconoce son PER (personas), ORG (organizaciones), LOC (lugares), DATE (fechas), MONEY (cantidades monetarias)

Spark Processor Job

► Desplegar información

Este proceso automático toma la información enriquecida de los cursos y la deja ordenada para que pueda buscarse fácilmente. No se requiere intervención humana: el componente se ejecuta de forma periódica, limpia los datos y los publica, de modo que otros sistemas solo consumen información coherente y homogénea.

El Spark Processor Job se despliega como un CronJob de Kubernetes y ejecuta el pipeline definido en `Services/docker/SparkProcessorJob/app/functions.py`. Utiliza como parámetros las variables de entorno `URI_MONGODB` (destino en Atlas) y `VOLUMEN_PVC` (ruta del volumen compartido) y registra cada etapa con `logging` para que el monitoreo se realice desde los logs del contenedor. Su ejecución completa aborda los siguientes pasos operativos:

- **Ingesta controlada:** `createSession` inicializa Spark con la conexión a MongoDB y `read_augmented_data` carga la carpeta `augmented`, creando la vista `augmented_data` para habilitar consultas SQL durante la sesión.
- **Estandarización de textos:** `uppercase_first_letter` recorre todas las columnas de tipo string y `normalize_entities` se encarga de arreglos y estructuras anidadas, asegurando que cada texto comience con mayúscula sin perder información contextual.
- **Normalización temporal:** `format_dates_ddmmyyyy_sql` transforma `date_extracted` al formato `DD/MM/YYYY`, unificando criterios de reporting y control de versiones de los documentos procesados.
- **Resumen ejecutivo:** se expone el UDF `resumen_simple` mediante `summary`, que genera la columna `short-description` con un máximo de 140 caracteres conservando palabras completas, lo cual mejora el consumo en interfaces de búsqueda.
- **Contexto relacional:** `add_related_products` identifica coincidencias de entidades entre productos, elimina auto-referencias y deduplica resultados, dejando en `productos_relacionados` hasta 10 recomendaciones directamente utilizables por la UI.
- **Publicación confiable:** `save_to_mongodb` persiste el resultado final en la colección `documents` de MongoDB Atlas con modo `overwrite`, garantizando que la versión más reciente del dataset esté disponible para Atlas Search.

De esta manera, cada ejecución del CronJob entrega datos limpios, resumidos y enriquecidos con relaciones, listos para ser indexados y consumidos por el resto de la plataforma.

Configuración Firestore

► Desplegar información

Credenciales de Firebase (`lib/firebase.ts`):

- **API Key:** `AIzaSyBGqepgUpRN9jftFRnzSMiFOnXEEzSJ8VU` – Clave para autenticación en cliente.
- **Auth Domain:** `proyect-db-2-itcr.firebaseio.com` – Dominio para auth.
- **Project ID:** `proyect-db-2-itcr` – ID del proyecto.
- **Storage Bucket:** `proyect-db-2-itcr.firebaseiostorage.app` – Para archivos.
- **Messaging Sender ID:** `121361936291` – Para notificaciones.
- **App ID:** `1:121361936291:web:c0d99e83a723641fa3b416` – ID de la app web.
- **Measurement ID:** `G-DZ3Q95442D` – Para Analytics.

Configuración:

- Inicializa Firebase con `initializeApp(firebaseConfig)`.
- Exporta `auth = getAuth(app)` para autenticación cliente.
- Exporta `db = getFirestore(app)` para Firestore.
- Crea colección `users` en Firestore Console para perfiles

Evidencias:



Configuración Mongo Atlas

► Desplegar información

Índice Atlas Search **default**

- **Colección:** **ecomm.documents**
- **Objetivo:** habilitar búsqueda full-text con facets y highlighting sobre los productos normalizados por Spark.
- Nota: Se asignaron de tipo facet las características mas relevantes que seran utilizadas.
- **Mapping utilizado:**

```
{
  "mappings": {
    "dynamic": true,
    "fields": {
      "authorComment": {
        "type": "string"
      },
      "currency": [
        {
          "analyzer": "lucene.keyword",
          "searchAnalyzer": "lucene.keyword",
          "type": "string"
        },
        {
          "type": "stringFacet"
        }
      ],
      "date_extracted": {
        "type": "date"
      },
      "description": {
        "type": "string"
      },
      "entities": {
        "fields": {
          "type": [
            {
              "analyzer": "lucene.keyword",
              "searchAnalyzer": "lucene.keyword",
              "type": "string"
            },
            {
              "type": "stringFacet"
            }
          ]
        }
      }
    }
  }
}
```

```
        "value": [
          {
            "type": "string"
          },
          {
            "type": "stringFacet"
          }
        ]
      },
      "type": "document"
    },
    "estimated_weeks": [
      {
        "type": "number"
      },
      {
        "type": "numberFacet"
      }
    ],
    "general_category": [
      {
        "type": "string"
      },
      {
        "analyzer": "lucene.keyword",
        "type": "autocomplete"
      },
      {
        "type": "stringFacet"
      }
    ],
    "language": [
      {
        "type": "string"
      },
      {
        "type": "stringFacet"
      }
    ],
    "price": [
      {
        "type": "number"
      },
      {
        "type": "numberFacet"
      }
    ],
    "productos_relacionados": {
      "fields": {
        "entities": {
          "fields": {
            "type": {
              "type": "stringFacet"
            }
          }
        }
      }
    }
  }
}
```

```

        },
        "type": "document"
    }
},
    "type": "document"
},
"rating_value": [
    {
        "type": "number"
    },
    {
        "type": "numberFacet"
    }
],
"reviews": {
    "dynamic": true,
    "fields": {
        "comment": {
            "type": "string"
        },
        "rating": [
            {
                "type": "number"
            },
            {
                "type": "numberFacet"
            }
        ]
    },
    "type": "document"
},
"short-description": {
    "type": "string"
},
"students": [
    {
        "type": "number"
    },
    {
        "type": "numberFacet"
    }
],
"title": {
    "type": "string"
}
}
}
}

```

Facets configurados

- `currency`, `general_category`, `language`, `specific_category`: facets de texto para filtros de navegación.
- `price`, `rating_value`, `students`, `reviews.rating`: facets numéricos que permiten agrupar resultados por rangos.

Highlighting

- Las consultas `$search` incluyen `highlight` sobre `title`, `description`, `short-description` y `reviews.comment`, devolviendo coincidencias resaltadas (`searchHighlights`).

Verificación

1. **Mapping:** revisado en Atlas UI → Search Indexes → `default` → JSON.
2. **Facets:** consultas `$searchMeta` retornan buckets para categorías y rangos numéricos verificando los `stringFacet` y `numberFacet` definidos.
3. **Highlighting:** consultas `$search` en Data Explorer muestran texto marcado en los campos configurados, cumpliendo el requisito de resaltado.

Colección ingestion

- **Colección:** `ecommerce.ingestion`
- **Objetivo:** almacenar el registro de todos los documentos HTML provenientes del bucket S3 antes de su procesamiento por el parser. Esta colección actúa como el punto de control del pipeline de ingesta registrando el estado `new|modified`
- **Esquema utilizado:**

```
{
  "_id": "",
  "fileName": "",
  "sizeBytes": 0,
  "md5": "",
  "state": "",
  "publishedAt": "",
  "processing": "",
  "processingError": "",
  "entitiesExtraction": "",
  "entitiesExtractionError": ""
}
```

Configuración RabbitMQ

► Desplegar información

Es el sistema que sirve como intermediario para que los servicios se pasen mensajes y realizar sus funciones correspondientes. Permite que el controller, BeautifulSoup y Spacy Entity Extractor trabajen de forma asíncrona, equilibrada en caso de que haya más de un Spacy Entity Extractor y soportando reinicio en caso de que algún mensaje no se procese. El pipeline utiliza 2 colas:

- **queue_parse**

Es producida por el Controller y consumida por el BeautifulSoup. Su propósito principal es notificar que hay archivos HTML nuevos o modificados en S3 que deben ser parseados.

- **queue_entity**

Es producida por el BeautifulSoup y consumida por el Spacy Entity Extractor. Su propósito principal es notificar que hay archivos JSON procesados listos para extracción de entidades.

UI

► Desplegar información

Uso de la AI

► Desplegar información

Para la realización del frontend de esta web, al ser desplegado en Vercel, se utilizó su propia AI llamado **v0 by Vercel**. Esto es una AI excelente para realizar frontend de sitios web, el cual hace los sitios con componentes muy estéticos y una paleta de colores agradable. También sirve un poco para el backend, pero su fuerte es el diseño del front. Los prompts que se utilizaron fueron los siguientes, estos fueron para crear el frontend usando MockUps, mientras los compañeros trabajaban para tener los campos correctos de los datos extraídos:

- Debo crear la UI de login y register en mi proyecto de NextJS usando Tailwind. Manejando siempre las mejores prácticas y los componentes más modernos, también se piensa utilizar firebase y firestore para la authentication
- Ahora crea una pantalla luego del proceso de Auth. Esta pantalla es para obtener cursos disponibles y realizar una búsqueda de algún curso, también debe tener una opción de búsqueda avanzada con estos facets mientras tanto "programación", "cocina", "creativo", "salud", "negocio", "deporte", "psicología", "ciencia", "cloud computing", "mantenimiento", "moda", "arte", "idiomas", "marketing. Recuerda hacerla moderna, con componentes modernos y para nextjs. Agregale un navbar bonito donde está ubicado el cerrar sesión
- Perfecto, ahora crea la pantalla al presionar un curso. Esta debe ser moderna, donde me va a incluir la información del curso específico como descripción, precio, idioma, certificación, reviews y otros campos que consideres correctos.

Reflexión

► Desplegar información

- Se ahorra bastante el tiempo, ya que el frontend suele durar por todos los componentes que se le deben de agregar. Este tiempo puede ser aprovechado para otras funciones como el backend.
- La inteligencia artificial tiene mejores ideas de diseño para mejorar la experiencia del usuario.
- El uso de paleta de colores y componentes son muy bonitos y minimalistas, aumentando el profesionalismo de la web

Rest API

- Desplegar información

Endpoints

- Ver endpoints
- Autenticación

Register

```
POST /api/register
```

Descripción: Crea un nuevo usuario en Firebase Auth y guarda su perfil en Firestore. Requiere email y contraseña válidos. La contraseña se maneja internamente por Firebase.

Ejemplo de request:

```
POST /api/register
Content-Type: application/json
{
  "email": "usuario@example.com",
  "password": "12345678"
}
```

Response (200 OK):

```
{
  "message": "Usuario registrado exitosamente",
  "uid": "H0ngPYZUUKUmijffu3zd7t8NPjE2"
}
```

Login

```
POST /api/auth/login
```

Descripción: Verifica el token ID enviado por el cliente (con Firebase Auth, el cual maneja verifica correo y contraseña) y setea una cookie de autenticación si es válido. También verifica que el usuario exista en Firestore.

Ejemplo de request:


```
POST /api/auth/login
Content-Type: application/json
{
  "token": "eyJhbGciOiJSUzI1NiIsImtpZCI6..."
}
```

Response (200 OK):

```
{
  "success": true,
  "uid": "H0ngPYZUUKUmijffu3zd7t8NPjE2"
}
```

Logout

```
POST /api/auth/logout
```

Descripción: Elimina la cookie de autenticación auth-token, cerrando la sesión del usuario.

Ejemplo de request:

```
POST /api/auth/logout
Content-Type: application/json
```

Response (200 OK):

```
{
  "success": true
}
```

► Cursos**Obtener cursos**

```
GET /api/courses
```

Descripción: Obtiene una lista de cursos desde la base de datos con filtros opcionales, ordenamiento, paginación y estadísticas agrupadas (facets). Si se envía el parámetro search, se utiliza Atlas Search para realizar búsquedas de texto con relevancia y resultados destacados (highlighting).

Parámetros de consulta (Query Params)

Parámetro	Tipo	Descripción
search	string	Texto a buscar en los campos <code>title</code> , <code>description</code> , <code>short-description</code> , <code>authorComment</code>
category	string	Filtra por categoría general del curso
estimatedWeeks	number	Filtra por semanas estimadas de duración
language	string	Filtra por idioma
currency	string	Filtra por tipo de moneda
studentsRange	string	Rango de estudiantes, formato <code>"min-max"</code> (ej. <code>"100-5000"</code>)
entityType	string	Filtra por tipo de entidad (<code>entities.type</code>)
entityValue	string	Filtra por valor de entidad (<code>entities.value</code>)
sortBy	string	Campo de ordenamiento (ej. <code>"rating_value"</code> , <code>"price"</code>)
order	string	Orden ascendente o descendente (<code>asc</code> o <code>desc</code>)
limit	number	Límite de resultados por página
page	number	Página actual

Ejemplo de request:

```
GET /api/courses?
search=javascript&category=Programación&language=es&limit=5&page=1
```

Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "_id": "64b12345f9c12f7a9c3e1111",
      "title": "Curso de JavaScript",
      "description": "Aprende JavaScript desde cero",
      "price": 49,
      "rating_value": 4.5,
      "students": 1050,
      "language": "es",
      "currency": "USD",
      "entities": [
        { "type": "plataforma", "value": "Edutin" }
      ],
      "highlights": [ /* fragmentos destacados si hay búsqueda */ ]
    }
  ]
}
```

```
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 10,
    "totalCount": 200,
    "limit": 20,
    "hasNextPage": true,
    "hasPrevPage": false
  },
  "facets": {
    "categories": [{ "name": "Programación", "count": 120 }],
    "estimatedWeeks": [{ "name": 8, "count": 30 }],
    "languages": [{ "name": "es", "count": 180 }],
    "currencies": [{ "name": "USD", "count": 150 }],
    "priceRange": {
      "buckets": [{ "range": 0, "count": 20 }, { "range": 50, "count": 100 }]
    },
    "ratingDistribution": [
      { "_id": 3, "count": 15 },
      { "_id": 4, "count": 80 },
      { "_id": 5, "count": 50 }
    ],
    "studentsRange": {
      "buckets": [{ "range": 0, "count": 10 }, { "range": 1000, "count": 50 }]
    },
    "entityTypes": [{ "name": "plataforma", "count": 2 }],
    "entityValues": [{ "name": "Edutin", "count": 50 }]
  },
  "filters": {
    "search": "javascript",
    "category": "Programación",
    "language": "es",
    "sortBy": "rating_value",
    "order": "desc"
  }
}
```

Obtener curso específico

```
GET /api/courses/{id}
```

Descripción: Obtiene la información detallada de un curso específico por su ID de MongoDB. Si se incluye el parámetro search, se utiliza Atlas Search para aplicar búsqueda semántica dentro del curso y resaltar coincidencias (highlighting). Además, devuelve productos relacionados y estadísticas de reseñas.

Parámetros de consulta (Query Params)

Parámetro	Tipo	Descripción
-----------	------	-------------

Parámetro	Tipo	Descripción
id	string	ID del curso en formato MongoDB ObjectId
search	string	Texto para aplicar búsqueda contextual con Atlas Search y resaltar coincidencias

Ejemplo de request:

```
GET /api/courses/64b12345f9c12f7a9c3e1111?search=javascript
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "_id": "64b12345f9c12f7a9c3e1111",
    "title": "Curso de JavaScript",
    "description": "Aprende a programar en JavaScript desde cero.",
    "price": 49,
    "rating_value": 4.5,
    "students": 1050,
    "language": "es",
    "currency": "USD",
    "general_category": "Programación",
    "productos_relacionados": [
      {
        "title": "Curso de HTML",
        "price": 29,
        "currency": "USD"
      }
    ],
    "reviewStats": {
      "totalReviews": 150,
      "averageRating": 4.5,
      "ratingDistribution": {
        "5": 100,
        "4": 30,
        "3": 10,
        "2": 5,
        "1": 5
      }
    },
    "relatedProducts": [
      {
        "id": "64b99999f9c12f7a9c3e7777",
        "title": "Curso de HTML",
        "price": 29,
        "currency": "USD",
        "rating_value": 4.6,
        "students": 800,
      }
    ]
  }
}
```

```

    "language": "es",
    "short_description": "Aprende los fundamentos del lenguaje HTML...",
    "isRelated": true,
    "originalCourseId": "64b99999f9c12f7a9c3e7777"
  }
],
"highlights": [
  {
    "path": "description",
    "texts": [
      { "value": "Aprende a programar en ", "type": "text" },
      { "value": "JavaScript", "type": "hit" },
      { "value": " desde cero.", "type": "text" }
    ]
  }
]
}
}

```

Método de seguridad implementado

► Desplegar información

Se utilizó el SDK de Firebase Admin para implementar manejo de seguridad, verificando que solo usuarios logueados accedan a recursos.

Middleware de Autenticación

- Configurado en `middleware.ts` con `matcher: ['/', '/courses/:path*']` para proteger página principal y de curso específico.
- Verifica la presencia y validez del token ID de Firebase usando `authAdmin.verifyIdToken()`.
- Si el token falta o es inválido:
 - Redirige a `/auth/login`.
 - Elimina la cookie `auth-token`.

Verificación de Tokens en Endpoints API

- En `/api/courses/route.ts` y `/api/courses/[id]/route.ts`:
 - Se extrae el token con `request.cookies.get('auth-token')`.
 - Se valida con `authAdmin.verifyIdToken(token)`.
 - Si la validación falla, no se ejecuta el endpoint.

Flujo General de Seguridad

1. El usuario inicia sesión y obtiene un ID Token de Firebase Auth.
2. El token se almacena en una cookie `auth-token`.
3. El middleware verifica el token en rutas protegidas para redirecciones.
4. Los endpoints verifican el token en cada solicitud para autorizar acceso a datos.
5. Si cualquier verificación falla, se deniega el acceso con errores apropiados.

Conclusiones

► Desplegar información

1. El Spark Processor centraliza la normalización del dataset en una sola ejecución hace en mayúscula textos, formatea fechas, genera resúmenes y construye relaciones basadas en entidades, dejando la colección documents lista para indexarse en Atlas Search.
2. Gracias al empaquetado de componentes como CronJob con imagen Docker parametrizable por Helm, el componente puede reejecutarse de forma controlada ante nuevas ingestas sin afectar a otros microservicios.
3. La implementación del scraping permitió obtener datos actualizados de forma automatizada, asegurando la repetición, escalabilidad y adaptabilidad a distintos sitios web.
4. Se ganaron conocimientos prácticos sobre web scraping, manipulación de archivos, carga mediante CLI y buenas prácticas de registro de errores.
5. La forma en que se diseña el controller evita que se procesen archivos que ya están en la base de datos y no han cambiado, brindando ahorro de recursos y tiempo de procesamiento.
6. La utilización de Spacy logra identificar correctamente entidades importantes sobre los productos de cursos, brindando así una mejor organización de la información dentro de cada uno de los productos que se mostrarán
7. El uso de Next.js para el despliegue en Vercel simplifica mucho el desarrollo y la implementación de aplicaciones full-stack. Las API routes permiten crear un backend serverless integrado, reduciendo la complejidad de configuración y facilitando la interacción con bases de datos como MongoDB Atlas.
8. Firebase facilita la autenticación de usuarios con herramientas listas para producción, eliminando la necesidad de desarrollar endpoints personalizados. Su integración con NextJS permite construir aplicaciones seguras en menos tiempo, manteniendo buenas prácticas en la gestión de sesiones y credenciales.
9. La biblioteca Beatiful Soup provee una serie de utilidades muy importantes para parsear archivos, permitiendo limpiar y normalizar etiquetas y documentos con formato HTML en este caso.
10. La utilización de un volumen compartido tipo ReadWriteMany permite que varios pods puedan leer y escribir en un mismo almacenamiento al mismo tiempo, permitiendo que varios componentes utilicen el volumen y siendo eficiente para flujos grandes.

Recomendaciones

► Desplegar información

1. Uso de variables de entorno Se recomienda centralizar la configuración del sistema mediante variables de entorno, lo cual facilita la mantenibilidad y portabilidad de la aplicación. Estas variables deben incluir, entre otros aspectos, las credenciales y parámetros de conexión a la base de datos, así como las direcciones y claves necesarias para el consumo de endpoints externos.

2. Añadir métricas y alertas simples (por ejemplo, contador de registros procesados y tiempo de ejecución) para detectar rápidamente anomalías en pipelines futuros y facilitar el monitoreo en producción.
3. Agregar un proceso de validación de los datos extraídos para asegurar calidad, eliminar duplicados y facilitar etapas de análisis.
4. Se recomienda implementar mejores prácticas de seguridad para el manejo de credenciales para evitar fallos en la seguridad del proyecto.
5. Se recomienda validar que los archivos HTML realmente contengan información para de esta manera evitar que el componente que los consume procese archivos innecesarios y mantener la calidad de los datos.
6. Se recomienda analizar bien el contexto en el que será utilizado el modelo Spacy, ya que este contiene modelos con diferentes características, algunos consumen más memoria lo cual puede ser no tan factible cuando se tienen recursos limitados, otros son más ligeros pero tienen peor rendimiento, por esto se requiere un análisis de cuál podría ser el más adecuado.
7. Para las instrucciones de este proyecto, donde se debe de desplegar una web en Vercel, se recomienda utilizar Next.js. Este framework es muy sencillo de desplegar, ya que es creado por el propio Vercel. Además, el uso del backend es facilitado por las API routes de Next.js, que permiten crear endpoints serverless directamente en el proyecto, sin preocuparse por desplegarlo en otro sitio.
8. Si se tiene la posibilidad, se recomienda el uso de Firebase para el manejo de autenticación de usuarios. Estas funcionalidades que ofrece facilitan mucho los procesos de registro, login, verificación de correos electrónicos, recuperación de contraseñas y gestión de sesiones, sin necesidad de implementar un backend personalizado para autenticación.
9. Se recomienda, a la hora de trabajar con componentes grandes o pensando en escalabilidad, parametrizar las métricas de cada componente para la instalación, permitiendo subir la cantidad de réplicas en caso de que la ejecución esté siendo inestable.
10. Se recomienda a la hora de parsear archivos html buscar los distintos lados donde viene la misma información para tener fallbacks, esto permite tener respuesta a errores o inconsistencias de los caracteres HTML.

Referencias

► Desplegar información

<https://spark.apache.org/docs/latest/api/python/index.html>

<https://www.mongodb.com/docs/spark-connector/current/>

<https://www.mongodb.com/products/platform/atlas-search>

<https://www.mongodb.com/docs/atlas/atlas-search/define-field-mappings/>

<https://www.mongodb.com/docs/atlas/atlas-search/facet/>

- <https://www.mongodb.com/docs/atlas/atlas-search/highlighting/>
- <https://thunderbit.com/es/blog/python-scraping-tutorial-for-beginners>
- <https://requests.readthedocs.io/en/latest/user/quickstart/#custom-headers>
- <https://pythones.net/archivos-en-python-crear-guardar-files/>
- <https://stackoverflow.com/questions/20986631/how-can-i-scroll-a-web-page-using-selenium-webdriver-in-python>
- <https://www.selenium.dev/selenium/docs/api/java/org/openqa/selenium/By.html>
- <https://stackoverflow.com/questions/69315951/how-to-run-aws-s3-sync-command-concurrently-for-different-prefixes-using-python/69317895#69317895>
- <https://www.codecademy.com/article/python-subprocess-tutorial-master-run-and-popen-commands-with-examples>
- <https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-configure.html>
- <https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html>
- <https://spacy.io/models/es>
- <https://spacy.io/usage/models>
- <https://beautiful-soup-4.readthedocs.io/en/latest/>
- <https://j2logo.com/python/web-scraping-con-python-guia-inicio-beautifulsoup/>
- <https://coderslegacy.com/10-most-important-functions-in-beautifulsoup/>

Tabla de Estado

► Desplegar información

Componente	% Evaluación	Estado	Observaciones
Controller	5 %	Implementado	Flujo de reconciliación y mensajería operativo.
Web Scraper	15 %	Implementado	Dataset de ≥500 productos almacenado en S3.
Beautiful Soup Parser	15 %	Implementado	Parsing HTML a JSON estable en despliegue de 2 réplicas.
spaCy Entity Extractor	10 %	Implementado	Entidades entities generadas y persistidas en augmented/.
Spark Processor Job	15 %	Implementado	Normalización completa y escritura en documents.

Componente	% Evaluación	Estado	Observaciones
Configuración Firestore / Mongo Atlas Search / RabbitMQ	10 %	Implementado	Servicios gestionados configurados y accesibles desde los microservicios.
UI y REST API	20 %	Implementado	UI en Vercel con autenticación y consumo de API segura.